

Decision Making in Python

So far, our programs have followed the same instructions every time they are run.

For example:

```
1  age = 20
2
3  print("Your age is", age)
```

The program always performs the same actions.

Real programs often need to make decisions.

For example:

- If a student's marks are 40 or more, the student has passed.
- If a person's age is 18 or more, they can vote.
- If the temperature is high, display a warning.
- If a password is correct, allow access.

Python allows us to make such decisions using **conditional statements**.

In this module, we will learn:

- What conditions are
- Comparison operators
- Boolean values
- The **if** statement
- The **else** statement
- The **elif** statement
- Indentation
- Logical operators
- Nested conditions
- Combining conditions
- Practical decision-making programs

What is a Condition?

A condition is something that can be evaluated as either **True** or **False**.

For example:

```
1 10 > 5
```

This condition is true.

Another example:

```
1 10 < 5
```

This condition is false.

Python represents these two possibilities using the Boolean values:

```
1 True
2 False
```

These values must begin with a capital letter.

```
1 True
2 False
```

They are not written as:

```
1 true
2 false
```

Comparison Operators

Comparison operators are used to compare values.

Operator	Meaning	Example
==	Equal to	10 == 10
!=	Not equal to	10 != 5
>	Greater than	10 > 5
<	Less than	5 < 10
>=	Greater than or equal to	10 >= 10
<=	Less than or equal to	5 <= 10

Equal To: ==

The == operator checks whether two values are equal.

```
1 print(10 == 10)
2 print(10 == 5)
```

Output:

```
1 True
2 False
```

The important point is that == compares values.

Assignment and Comparison Are Different

The single equals sign = is used for assignment.

For example:

```
1 age = 20
```

This stores 20 in the variable `age`.

The double equals sign == is used for comparison.

For example:

```
1 age == 20
```

This asks:

Is the value of `age` equal to 20?

Important Note: Confusing = and == is one of the most common beginner mistakes.

Not Equal To: !=

The != operator checks whether two values are different.

```
1 print(10 != 5)
2 print(10 != 10)
```

Output:

```
1 True
2 False
```

Greater Than: >

```
1 print(10 > 5)
2 print(5 > 10)
```

Output:

```
1 True
2 False
```

Less Than: <

```
1 print(5 < 10)
2 print(10 < 5)
```

Output:

```
1 True
2 False
```

Greater Than or Equal To: >=

```
1 print(10 >= 10)
2 print(10 >= 5)
3 print(5 >= 10)
```

Output:

```
1 True
2 True
3 False
```

The expression `10 >= 10` is true because 10 is equal to 10.

Less Than or Equal To: `<=`

```
1 print(5 <= 10)
2 print(10 <= 10)
3 print(15 <= 10)
```

Output:

```
1 True
2 True
3 False
```

The if Statement

The `if` statement allows a program to execute a block of code only when a condition is true.

Basic structure:

```
1 if condition:
2     statement
```

Example:

```
1 age = 20
2
3 if age >= 18:
4     print("You are an adult.")
```

Output:

```
1 You are an adult.
```

If the condition is false, the indented statement is not executed.

For example:

```
1 age = 15
2
```

```
3 if age >= 18:
4     print("You are an adult.")
```

There is no output because the condition is false.

Indentation

Python uses indentation to show which statements belong to a block of code.

For example:

```
1 age = 20
2
3 if age >= 18:
4     print("You are an adult.")
5     print("You can make your own decisions.")
```

Both `print()` statements belong to the `if` block because they are indented.

The standard practice is to use four spaces for indentation.

Do not write:

```
1 age = 20
2
3 if age >= 18:
4 print("You are an adult.")
```

The second line should be indented.

Important Note: Indentation is part of Python's syntax. Incorrect indentation can cause an error or change how the program behaves.

Using if With User Input

Conditions become more useful when the program receives information from the user.

Example:

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("You are an adult.")
```

If the user enters 20:

```
1 You are an adult.
```

If the user enters 15, nothing is displayed because the condition is false.

The else Statement

Sometimes we want the program to do one thing when a condition is true and another thing when it is false.

We can use `else`.

Example:

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("You are an adult.")
5 else:
6     print("You are a minor.")
```

If the user enters 20:

```
1 You are an adult.
```

If the user enters 15:

```
1 You are a minor.
```

The general structure is:

```
1 if condition:
2     statement1
3 else:
4     statement2
```

Only one of the two blocks is executed.

The elif Statement

Sometimes there are more than two possible outcomes.

For example, suppose marks determine a student's grade.

We might use:

```
1 marks = int(input("Enter marks: "))
2
3 if marks >= 80:
```

```
4     print("Grade A")
5 elif marks >= 60:
6     print("Grade B")
7 elif marks >= 40:
8     print("Grade C")
9 else:
10    print("Fail")
```

If the user enters 85:

```
1  Grade A
```

If the user enters 65:

```
1  Grade B
```

If the user enters 45:

```
1  Grade C
```

If the user enters 30:

```
1  Fail
```

The conditions are checked from top to bottom.

Once Python finds a true condition, its block is executed and the remaining `elif` and `else` blocks are skipped.

The Importance of Condition Order

Consider:

```
1  marks = 85
2
3  if marks >= 40:
4      print("Pass")
5  elif marks >= 80:
6      print("Grade A")
```

The output is:

```
1  Pass
```

Why?

Because 85 is greater than or equal to 40, so Python executes the first block and stops checking the remaining conditions.

A better order is:

```
1 marks = 85
2
3 if marks >= 80:
4     print("Grade A")
5 elif marks >= 40:
6     print("Pass")
7 else:
8     print("Fail")
```

Output:

```
1 Grade A
```

Important Note: When conditions overlap, the order in which they are written matters.

Boolean Expressions

A comparison produces a Boolean result.

For example:

```
1 age = 20
2
3 print(age >= 18)
```

Output:

```
1 True
```

We can store Boolean results in variables:

```
1 age = 20
2
3 adult = age >= 18
4
5 print(adult)
```

Output:

```
1 True
```

The variable `adult` contains the Boolean value `True`.

Logical Operators

Logical operators allow us to combine conditions.

Python has three important logical operators:

Operator	Meaning
<code>and</code>	Both conditions must be true
<code>or</code>	At least one condition must be true
<code>not</code>	Reverses a Boolean value

The and Operator

The `and` operator requires both conditions to be true.

Example:

```
1 age = 20
2 marks = 75
3
4 if age >= 18 and marks >= 40:
5     print("Eligible")
```

Both conditions are true:

- Age is at least 18.
- Marks are at least 40.

Therefore the output is:

```
1 Eligible
```

If either condition is false, the complete condition becomes false.

The or Operator

The `or` operator is true when at least one condition is true.

Example:

```
1 day = "Sunday"
2
3 if day == "Saturday" or day == "Sunday":
4     print("Weekend")
```

Output:

```
1 Weekend
```

The not Operator

The **not** operator reverses a Boolean value.

For example:

```
1 is_raining = False
2
3 print(not is_raining)
```

Output:

```
1 True
```

Another example:

```
1 age = 20
2
3 if not age < 18:
4     print("Adult")
```

Output:

```
1 Adult
```

Combining Multiple Conditions

We can combine several comparisons.

Example:

```
1 age = 25
2 marks = 80
3
```

```
4 if age >= 18 and marks >= 50:
5     print("Eligible")
```

We can also use parentheses when they make the condition easier to understand.

```
1 age = 20
2 course = "Python"
3
4 if age >= 18 and course == "Python":
5     print("Eligible")
```

Comparing Strings

Conditions can also compare strings.

Example:

```
1 course = input("Enter your course: ")
2
3 if course == "Python":
4     print("You are learning Python.")
```

String comparison is case-sensitive.

For example:

```
1 course = "python"
2
3 print(course == "Python")
```

Output:

```
1 False
```

We can use `lower()` to make the comparison case-insensitive:

```
1 course = input("Enter your course: ")
2
3 if course.lower() == "python":
4     print("You are learning Python.")
```

Using in in Conditions

The `in` operator can be used inside conditions.

Example:

```
1 message = input("Enter a message: ")
2
3 if "Python" in message:
4     print("The word Python was found.")
```

This is useful when we need to search for something in user input.

Nested if Statements

An **if** statement can appear inside another **if** statement.

This is called a **nested condition**.

Example:

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("You are an adult.")
5
6     if age >= 60:
7         print("You are a senior citizen.")
```

If the user enters 65, the output is:

```
1 You are an adult.
2 You are a senior citizen.
```

The second condition is checked only if the first condition is true.

Practical Example: Pass or Fail

Consider a simple student result program.

```
1 marks = int(input("Enter marks: "))
2
3 if marks >= 40:
4     print("Pass")
5 else:
6     print("Fail")
```

If marks are 75:

```
1 Pass
```

If marks are 32:

```
1 Fail
```

Practical Example: Even or Odd

We can use the remainder operator together with a condition.

A number is even when its remainder after division by 2 is zero.

```
1 number = int(input("Enter a number: "))
2
3 if number % 2 == 0:
4     print("Even")
5 else:
6     print("Odd")
```

For example, if the user enters 12:

```
1 Even
```

If the user enters 15:

```
1 Odd
```

Practical Example: Largest of Two Numbers

We can compare two numbers.

```
1 a = int(input("Enter first number: "))
2 b = int(input("Enter second number: "))
3
4 if a > b:
5     print("First number is larger.")
6 elif b > a:
7     print("Second number is larger.")
8 else:
9     print("Both numbers are equal.")
```

Practice Exercises

Attempt each exercise yourself before looking at the answers.

Exercise 1: Positive Number

Ask the user to enter a number.

Display **Positive** if the number is greater than zero.

Exercise 2: Adult or Minor

Ask the user to enter their age.

Display **Adult** if the age is 18 or above. Otherwise display **Minor**.

Exercise 3: Pass or Fail

Ask the user to enter marks.

Display **Pass** if the marks are 40 or above. Otherwise display **Fail**.

Exercise 4: Even or Odd

Ask the user to enter an integer.

Display whether it is even or odd.

Exercise 5: Number Comparison

Ask the user to enter two numbers.

Display which number is larger. If they are equal, display **Both are equal**.

Exercise 6: Grade

Ask the user to enter marks.

Use the following grading system:

Marks	Grade
80 or above	A
60 to 79	B
40 to 59	C
Below 40	Fail

Exercise 7: Voting Eligibility

Ask the user to enter their age.

Display `Eligible to vote` if the age is 18 or above. Otherwise display `Not eligible to vote`.

Exercise 8: Username

Ask the user to enter a username.

If the username is `admin`, display `Welcome Admin`. Otherwise display `Welcome User`.

Exercise 9: Password Check

Ask the user to enter a password.

Use:

```
1 password = "python123"
```

If the entered password matches, display `Access Granted`. Otherwise display `Access Denied`.

Exercise 10: Weekend Check

Ask the user to enter a day.

If the day is Saturday or Sunday, display `Weekend`.

Otherwise display `Weekday`.

Make the comparison work even if the user enters the day in lowercase.

Exercise 11: Admission Eligibility

Ask the user for:

- Age
- Marks

The student is eligible if:

- Age is at least 18.
- Marks are at least 50.

Use the **and** operator.

Exercise 12: Discount Eligibility

Ask the user for the purchase amount.

If the amount is at least 5000, display:

```
1  You are eligible for a discount.
```

Otherwise display:

```
1  No discount available.
```

Exercise 13: Temperature

Ask the user to enter the temperature.

Use the following rules:

- 35 or above: Very Hot
- 25 to 34: Hot
- 15 to 24: Moderate
- Below 15: Cold

Exercise 14: Divisibility

Ask the user for a number.

Check whether the number is divisible by both 3 and 5.

Display:

```
1  Divisible by both 3 and 5
```

or:

```
1  Not divisible by both 3 and 5
```

Exercise 15: Word Search

Ask the user to enter a sentence.

Check whether the word `Python` occurs in the sentence.

Make the search case-insensitive.

Exercise 16: Positive, Negative, or Zero

Ask the user to enter a number.

Display:

- Positive
- Negative
- Zero

depending on the number entered.

Answers to Practice Exercises

The answers below are provided for checking your work after attempting the exercises yourself.

Answer 1: Positive Number

```
1 number = float(input("Enter a number: "))
2
3 if number > 0:
4     print("Positive")
```

Answer 2: Adult or Minor

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("Adult")
5 else:
6     print("Minor")
```

Answer 3: Pass or Fail

```
1 marks = float(input("Enter marks: "))
2
3 if marks >= 40:
4     print("Pass")
5 else:
6     print("Fail")
```

Answer 4: Even or Odd

```
1 number = int(input("Enter an integer: "))
2
3 if number % 2 == 0:
4     print("Even")
5 else:
6     print("Odd")
```

Answer 5: Number Comparison

```
1 a = float(input("Enter first number: "))
2 b = float(input("Enter second number: "))
3
4 if a > b:
5     print("First number is larger.")
6 elif b > a:
7     print("Second number is larger.")
8 else:
9     print("Both are equal.")
```

Answer 6: Grade

```
1 marks = float(input("Enter marks: "))
2
3 if marks >= 80:
4     print("Grade A")
5 elif marks >= 60:
```

```
6     print("Grade B")
7 elif marks >= 40:
8     print("Grade C")
9 else:
10    print("Fail")
```

Answer 7: Voting Eligibility

```
1 age = int(input("Enter your age: "))
2
3 if age >= 18:
4     print("Eligible to vote")
5 else:
6     print("Not eligible to vote")
```

Answer 8: Username

```
1 username = input("Enter username: ")
2
3 if username == "admin":
4     print("Welcome Admin")
5 else:
6     print("Welcome User")
```

Answer 9: Password Check

```
1 password = input("Enter password: ")
2
3 if password == "python123":
4     print("Access Granted")
5 else:
6     print("Access Denied")
```

Answer 10: Weekend Check

```
1 day = input("Enter a day: ")
2
3 day = day.strip().lower()
4
5 if day == "saturday" or day == "sunday":
6     print("Weekend")
7 else:
8     print("Weekday")
```

Answer 11: Admission Eligibility

```
1 age = int(input("Enter age: "))
2 marks = float(input("Enter marks: "))
3
4 if age >= 18 and marks >= 50:
5     print("Eligible for admission")
6 else:
7     print("Not eligible for admission")
```

Answer 12: Discount Eligibility

```
1 amount = float(input("Enter purchase amount: "))
2
3 if amount >= 5000:
4     print("You are eligible for a discount.")
5 else:
6     print("No discount available.")
```

Answer 13: Temperature

```
1 temperature = float(input("Enter temperature: "))
2
3 if temperature >= 35:
4     print("Very Hot")
5 elif temperature >= 25:
6     print("Hot")
```

```
7 elif temperature >= 15:
8     print("Moderate")
9 else:
10    print("Cold")
```

Answer 14: Divisibility

```
1 number = int(input("Enter a number: "))
2
3 if number % 3 == 0 and number % 5 == 0:
4     print("Divisible by both 3 and 5")
5 else:
6     print("Not divisible by both 3 and 5")
```

Answer 15: Word Search

```
1 sentence = input("Enter a sentence: ")
2
3 if "python" in sentence.lower():
4     print("Python was found.")
5 else:
6     print("Python was not found.")
```

Answer 16: Positive, Negative, or Zero

```
1 number = float(input("Enter a number: "))
2
3 if number > 0:
4     print("Positive")
5 elif number < 0:
6     print("Negative")
7 else:
8     print("Zero")
```

Mini Project: Student Result Program

Create a program that asks the user for:

1. Student name
2. Marks in three subjects

The program should calculate:

- Total marks
- Average marks

Then determine the result.

The student passes only if the average is at least 40.

For students who pass, display a grade using:

Average	Grade
80 or above	A
60 to 79.99	B
40 to 59.99	C
Below 40	Fail

A sample run:

```
1 Enter student name: Rahul
2 Enter marks in Subject 1: 75
3 Enter marks in Subject 2: 82
4 Enter marks in Subject 3: 68
5
6 Student Name: Rahul
7 Total Marks: 225
8 Average Marks: 75.0
9 Grade: B
10 Result: Pass
```

Try to write the program yourself before looking at the answer.

Answer: Student Result Program

```
1  name = input("Enter student name: ")
2
3  mark1 = float(input("Enter marks in Subject 1: "))
4  mark2 = float(input("Enter marks in Subject 2: "))
5  mark3 = float(input("Enter marks in Subject 3: "))
6
7  total = mark1 + mark2 + mark3
8  average = total / 3
9
10 print()
11 print(f"Student Name: {name}")
12 print(f"Total Marks: {total}")
13 print(f"Average Marks: {average}")
14
15 if average >= 80:
16     print("Grade: A")
17     print("Result: Pass")
18 elif average >= 60:
19     print("Grade: B")
20     print("Result: Pass")
21 elif average >= 40:
22     print("Grade: C")
23     print("Result: Pass")
24 else:
25     print("Grade: Fail")
26     print("Result: Fail")
```

Final Practice Task

Create a simple login program.

The program should ask the user for:

- Username
- Password

Use these stored values:

```
1 correct_username = "admin"
2 correct_password = "python123"
```

The program should behave as follows:

If both username and password are correct:

```
1 Login successful.
2 Welcome Admin.
```

If the username is correct but the password is wrong:

```
1 Incorrect password.
```

If the username is wrong:

```
1 User not found.
```

Answer: Login Program

```
1 correct_username = "admin"
2 correct_password = "python123"
3
4 username = input("Enter username: ")
5 password = input("Enter password: ")
6
7 if username == correct_username:
8     if password == correct_password:
9         print("Login successful.")
10        print("Welcome Admin.")
11    else:
12        print("Incorrect password.")
13 else:
14    print("User not found.")
```

Module Review

Before moving to the next module, make sure you understand:

- What a condition is.

- What Boolean values are.
- How comparison operators work.
- The difference between `=` and `==`.
- How the `if` statement works.
- How indentation works in Python.
- How `else` works.
- How `elif` works.
- Why the order of conditions matters.
- How to compare strings.
- How the `and` operator works.
- How the `or` operator works.
- How the `not` operator works.
- How to combine multiple conditions.
- How to use `in` inside conditions.
- What nested conditions are.
- How to build simple decision-making programs.

End of Python Module 3